Easier UVM 代码生成器参考指南

Easier UVM Code Generator
Reference Guide

中文全译本 + Linux / VCS 实施附录

原作：Doulos

对应原网页版本：2016-04-18；用户提供的打印件：42 页

译文整理日期：2026-10-26

译文包含全部 264 条正文、标题与列表条目，保留 140 段代码/命令和 3 幅原图。

正文为原文翻译；末尾“实施附录”是针对 Linux + VCS 的补充建议，单独标识，不属于原文。代码、参数名、文件名、取值和版本日期保留英文；原图下方提供中文术语对照。

阅读说明

原件由网页打印为图片型 PDF。本译本对 42 页逐页进行了文字识别，并与对应的 Doulos 官方参考页交叉核对，以恢复代码、参数和长段说明。译文重新排版，页码与原文不同；各节注明原件页码范围。

代码和命令按原文保留，包括英文注释，以便与原发行包核对。层次图中的箭头由支持该字形的字体呈现；代码中的缩进和声明顺序保留。

正文忠实保留 2016 年文档使用的工具名和安装方式。原文要求以 root 安装 Perl 模块的内容只作为翻译保留；你的 Linux 工作服务器建议优先使用现有公司环境或用户目录，参见实施附录。

原文几处标识符或措辞存在疑点：trans_var_enum、req_seq_inc、agent_has env，以及 top_env_generate_methods_inside/after_class 的合写形式。译文保留原写法；实际配置应与所下载版本的脚本、帮助和示例核对，不能把这些疑点直接当成新选项。

原文对 reg_cover_inc 的覆盖组包含位置有历史说明；其后又明确要求包含名为 m_cov 的完整覆盖组。应以实际生成代码及对应版本示例为准。

英文	统一译法
agent / env	agent 组件 / 环境
transaction / sequence	事务 / 序列
sequencer	序列仲裁器
driver / monitor	驱动器 / 监视器
reference model / scoreboard	参考模型 / 记分板
test harness / covergroup	测试支架 / 覆盖组

原文对照：Doulos 官方参考指南

目录

01 关键概念 4

02 安装代码生成器 5

03 运行代码生成器 6

04 接口模板文件 7

05 寄存器模板文件 15

06 公共模板文件 16

07 通用模板文件 - 参考模型和记分板 21

08 通用模板文件 - 支持加速/硬件仿真的环境 23

09 引脚列表文件 23

10 生成的结构 25

11 编译和仿真 28

12 添加用户自定义序列 29

13 添加用户自定义寄存器序列 30

14 时钟和复位 30

15 对生成的代码进行最小扩展 30

16 最小 include 文件的规则 31

17 在类之前、类内部和类之后包含代码 32

18 链接 33

附录 A 在 Linux + VCS 工作服务器上落地 34

01 关键概念

原件页码：1-3

在深入了解细节之前，应先理解以下概念。

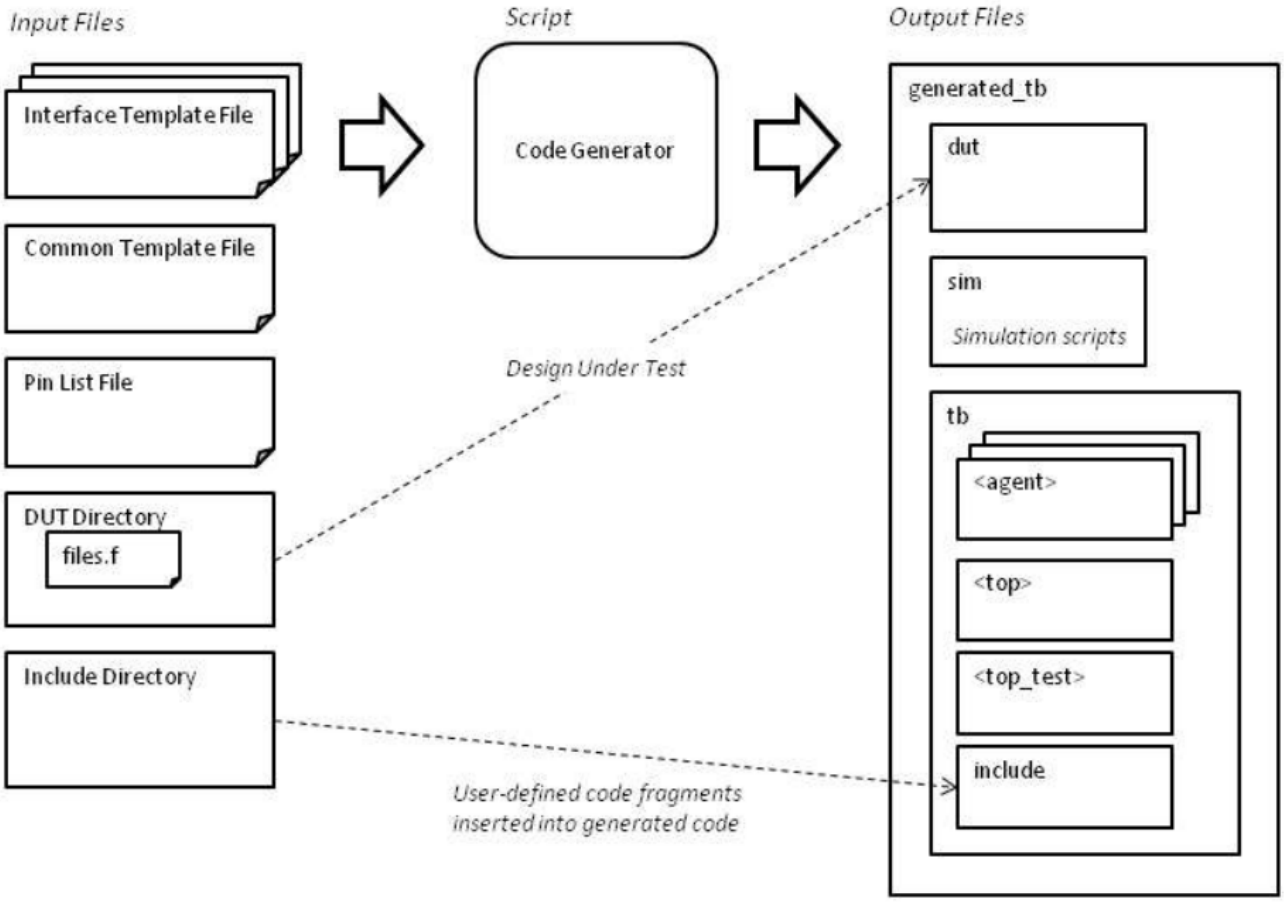
使用方式

代码生成器有三种可能的使用方式（参见《教程第 1 部分：入门》）：

- 仅作为学习工具，用来生成并运行可正常工作的 UVM 代码示例。
- 用来生成生产代码的第一个版本，此后手动维护生产代码，不再运行代码生成器。
- 随着项目推进，生成并重新生成生产代码中的样板代码，尽可能推迟需要开始手动维护样板代码的时点。

输入与输出

代码生成器的工作方式是：以一组模板文件、包含文件以及 DUT 代码作为输入，生成一组 SystemVerilog 源文件和仿真器脚本作为输出，并覆盖此前生成的所有文件。



图内术语：Input Files = 输入文件；Interface Template File = 接口模板文件；Common Template File = 通用模板文件；Pin List File = 引脚列表文件；DUT Directory = DUT 源目录；Include Directory = 用户包含文件目录；Code Generator = 代码生成器；Output Files = 输出文件；Simulation scripts = 仿真脚本。目录名和类名保持英文。

图：输入与输出

对外部文件的引用

输出文件是自包含的，但输入文件中引用的 SystemVerilog 包，以及输入文件中通过 `include 指令引用的文件除外。除非被引用的包和包含文件明确出现在输入文件集中，否则它们不会被复制到输出文件集中。因此，代码生成器不会解析 import 声明和 `include 指令背后的内容，而只是复制声明或指令本身，不复制关联文件。由此，生成的代码可以引用外部文件和包，包括 UVM 包本身。

接口模板文件

每个接口模板文件定义一个类型集，顾名思义，就是一个包中的一组 SystemVerilog 类（表示 UVM agent 组件、序列、配置、环境等）以及一个 SystemVerilog 接口。接口模板文件还可以指定该 agent 组件在单个环境中如何实例化的相关信息，即实例数量、agent 组件处于主动模式还是被动模式，以及寄存器模型中相应地址映射的位置。生成的 agent 组件除了可以实例化在自己的环境中，也可以实例化在其他环境或顶层环境中（参见“接口模板文件”）。

其他模板文件

除了接口模板文件，还有一个公共模板文件，用于指定同等适用于所有 agent 组件的设置，或适用于顶层模块、顶层环境和测试的设置（参见“公共模板文件”）。此外，还有一个 dut_pfile，用于指定生成的接口实例与 DUT 之间的引脚级连接（参见“引脚列表文件”）。

修改生成的代码

接口模板文件和公共模板文件提供了两类方法：将包含文件中的用户自定义代码片段插入生成的代码，以及在代码的特定位置禁止自动生成代码。这些用户自定义片段可以内联插入，也可以使用 `include 指令插入。无论采用哪种方式，用户自定义代码片段都会成为输出文件集的一部分（这与输入源代码中的 `include 指令不同，代码生成器不会解析后者所引用的内容）。

由于每次运行代码生成器都会覆盖输出文件集，因此必须注意，确保对输出文件集所做的任何修改都不会丢失。有两种好的方法：

- 使用模板文件中的设置来修改生成的代码。
- 编写一个脚本，在运行代码生成器之后立即对生成的文件集执行所需的编辑。

区分多个实例

接口模板文件中的所有设置，包括任何用户自定义代码片段，都会影响生成的类（类型集），因此会同等适用于这些类型的所有对象。给定类型集的多个实例将具有对称性，包括在 agent 组件上启动的默认序列。此外，之后对这些序列应用的所有工厂覆盖（将一个序列替换为另一个序列）也会对称地应用于所有实例，因为序列的工厂覆盖始终是类型覆盖（而 UVM 组件可以使用工厂实例覆盖来区分）。要区分同一个序列类的多个实例，唯一显而易见的方法是在顶层配置对象中赋予实例特有的信息，然后由序列对象读取这些信息。

如果需要同一 agent 组件 / 接口的多个完全相同或近乎相同的实例，最佳方法是使用 number_of_instances 设置。如果需要同一 agent 组件 / 接口的多个差异很大的实例（或在不同上下文中使用这些实例），最佳方法是为每个实例分别定义一个接口模板文件。

02 安装代码生成器

原件页码：3-4

代码生成器由单个 Perl 脚本 easier_uvm_gen.pl 组成。

需要安装 Perl 5.8.0 或更高版本，以及 File::Copy::Recursive、File::stat 和 Time::Piece 模块。可以输入以下命令来检查是否已安装这些模块：

```
perl -MFile::Copy::Recursive -e 1
perl -MFile::stat -e 1
```

如果已经安装该模块，命令应当执行完毕且不输出任何内容。如果未安装该模块，则会显示错误信息；此时可以以 root 身份运行以下命令来安装它：

```
perl -MCPAN -e 'install File::Copy::Recursive'
perl -MCPAN -e 'install File::stat'
```

2015-06 版本之前的早期代码生成器还要求执行以下操作，但现在已不再需要：

```
perl -MCPAN -e 'install Time::Piece'
```

03 运行代码生成器

原件页码：4-5

可以按以下方式运行代码生成器脚本：

```
perl easier_uvm_gen.pl [-t] <template> <template> ...
```

下列每个命令行开关都是可选的。

- t <template> 列出接口模板文件。尽管命令行中必须指定一个或多个模板文件名，但 -t 开关本身是可选的。建议按照惯例为模板文件使用 .tpl 扩展名，不过脚本实际上接受任何文件名。
- p <top> 指定前缀，用于构造与顶层验证环境关联的文件、模块、包和 UVM 类的名称。也可以使用公共模板文件中的 prefix 设置指定该前缀。默认值为 top。
- m <filename> 指定公共模板文件相对于当前目录的路径。默认值为 common.tpl。
- s <path > 指定 Syosil 记分板源文件的路径。此命令行开关覆盖公共模板文件中 syosil_scoreboard_src_path 设置的值。[2016-03-08]
- c 使代码生成器在发出警告后继续执行。默认情况下，代码生成器会在警告后显示 Continue? (y/n) 提示。
- nopack 禁止在 uvm_sequence_item 类中自动生成 do_pack 和 do_unpack 方法。此选项用于向后兼容早期版本的代码生成器脚本，因为这些早期版本不生成上述方法。
- h 打印此命令行开关列表，不执行其他操作。

生成的文件位于公共模板文件所指定的 <project>/tb 目录中（见下文）。根据命令行所给模板文件中的设置，顶层环境 <top>_env 可以实例化下层 UVM 环境或 agent 组件，并为每个实例创建一个配置对象。下层环境会实例化 agent 组件、覆盖率收集器组件和寄存器模型。每个 agent 组件会实例化一个序列仲裁器、一个驱动器和一个监视器。在创建 UVM 组件层次结构的同时，代码生成器还会创建一个 UVM 测试示例，在每个 agent 组件上运行一个简单序列。

每次运行代码生成器脚本，都会在 <project> 目录下生成一整套文件。这些文件是自包含的，除非用户提供的代码（包括 DUT 和验证环境的所有包含文件）恰好含有引用外部文件的嵌套 `include 指令；在这种情况下，这些外部文件不会被复制到 <project> 目录。如果计划日后重新生成代码，就不应修改生成的代码，因为再次运行生成器时，所有修改都会被覆盖。生成器提供了一种机制（如下文所述），可以将用户自定义代码插入生成的代码中。

04 接口模板文件

原件页码：5-17

每个 DUT 接口都必须有一个模板文件。模板文件定义该接口以及相应 UVM agent 组件及其事务所特有的变量和选项。每个模板文件采用以下格式：

以 # 字符开头的任何行都视为注释并被忽略。允许在行尾添加注释。当 = 右侧的值由多个条目组成时，各条目可以用逗号或空格分隔。模板文件中的行顺序在大多数情况下并不重要。

```
agent_name = <agent>
```

agent_name 用于构造与特定 agent 组件关联的所有 UVM 类和变量的名称。agent_name 应当是模板文件中的第一项设置。

```
trans_item = <name>
```

trans_item 用作 agent 组件所用事务（序列项）的类名：序列项和分析事务使用同一种事务类型。

```
number_of_instances = <integer>
```

如果存在，number_of_instances 指定给定 agent 组件 / 接口的实例数量。默认值为 1。如果 agent_has_env = no，这些 agent 组件实例将在顶层环境中生成；否则将在 agent 组件环境中生成。在 dut_pfile 中，应当使用后缀 _0、_1、_2、... 来区分同一接口的多个实例。

对于具有寄存器模型的 agent 组件 / 接口，agent 组件无论如何都会包含在自己的环境中。与该 agent 组件关联的寄存器子块，其在顶层寄存器块中的实例名称必须带有后缀 _0、_1、...。顶层寄存器块中关联映射的名称也必须带有相同的后缀。生成的序列 <agent>_env_default_seq 将具有 regmodel_0、regmodel_1、... 等成员（即带有相同后缀），供寄存器序列调用，以读写寄存器。

```
trans_var  = <declaration>
trans_var  = <declaration>
...
```

每条 trans_var 声明都会包含在生成的事务（序列项）类中。如此声明的每个变量都会纳入该类生成的 do_copy、do_compare、do_print、do_record、do_pack、do_unpack 和 convert2string 方法。每个 <declaration> 可以是一条完整的类属性（变量）声明或一条注释，例如 rand logic [7:0] data ;。trans_var 也可以定义用户自定义类型（typedef），或者 const 或 static 类属性；生成的 do_copy 和 do_compare 方法不会复制或比较这些内容。例如：

```
trans_var = typedef byte unsigned ubyte;
trans_var = rand ubyte data;
trans_var = const ubyte zero = 0;
trans_var = static int count;
```

将 trans_var 条目从模板文件复制到生成的代码时，会保留其格式，因此可以根据需要在模板文件中将变量名和 / 或初始值纵向对齐。变量将按照 trans_vars 给出的顺序进行打包和解包。

```
trans_enum_var = <declaration>                                    [2015-11]
trans_enum_var = <declaration>
...
```

对于枚举类型的变量，应使用此设置代替 trans_var，例如：

```
trans_enum_var = typedef enum {up, down} dir_t;
trans_enum_var = rand dir_t dir;
```

注意，枚举类型必须使用 typedef 单独定义，不能作为变量定义的一部分进行定义。typedef 可以作为 trans_var 或 trans_enum_var 给出。在生成的 do_pack 和 do_unpack 方法中，必须区分枚举变量与非枚

举变量；相应地，如果使用 -nopack 命令行开关禁止生成这些方法，就不必区分枚举变量。生成的 convert2string 方法会利用 trans_var_enum，将其转换为枚举值，但这并非严格必需（trans_var 的值按整数格式化）。

```
trans_meta      = <declaration>                                          [2015-11]
...
trans_enum_meta = <declaration>                                          [2015-11]
...
```

每条 trans_meta 和 trans_enum_meta 声明都会包含在生成的事务（uvm_sequence_item）类中，但如此声明的变量不会纳入生成的 do_compare、do_pack 和 do_unpack 方法。这些变量表示存在于每个事务对象中的元数据，但不表示发送到 DUT 或从 DUT 接收的数据。trans_enum_meta 设置仅与 convert2string 有关，因为元数据本来就不会被打包和解包。换句话说，trans_meta 与 trans_enum_meta 的唯一区别在于生成的 convert2string 方法如何格式化该变量的值。typedef 可以作为 trans_var、trans_enum_var、trans_meta 或 trans_enum_meta 给出，唯一的区别在于最终在事务类中的声明顺序。

trans_var、trans_enum_var、trans_meta 和 trans_enum_meta 设置都允许任意数量的打包数组维度，以及单个非打包数组维度。非打包维度必须指定为大小，而不能指定为范围。所有维度都可以用包含参数名的表达式给出。例如：

```
trans_var       = rand bit [7:0][7:0] square;    # Two packed dimensions
trans_var       = rand int unsigned table [16];  # One unpacked dimension        [2015-11]

trans_enum_meta = typedef enum logic [1:0] {red, green, blue} color_t;          [2015-11]
trans_enum_meta = color_t [N-1:0][N-1:0] lookup [M];                            [2015-11]
```

用于字段打包和解包的 UVM 宏不支持枚举类型数组。因此，如果要生成 do_pack 和 do_unpack 方法，trans_enum_var 就不允许包含非打包数组维度。（上例中的 trans_enum_meta 可以使用，是因为元数据本来就不纳入 do_pack / unpack 方法。）

自动生成的事务覆盖组中不包含非打包数组。

```
trans_var_constraint = <constraint_block>
trans_var_constraint = <constraint_block>
...
```

如果存在，每项 trans_var_constraint 都指定一个约束块，该约束块会作为约束包含在事务（序列项）类中，例如：

```
trans_var_constraint = { addr < data; data < 55; }
trans_var_constraint = { solve cmd before data; cmd < data; }
```

也可以使用 trans_var 指定约束。这样做的优点是可以显式命名约束，例如：

```
trans_var = constraint addr_lt_data    { addr < data; data < 55; }
trans_var = constraint cmd_before_data { solve cmd before data; cmd < data; }

config_var  = <declaration>
config_var  = <declaration>
...
```

每条 config_var 声明都会包含在生成的 agent 组件配置类中。也可以在公共模板文件中使用 config_var，将声明添加到顶层配置类中。参见 agent_copy_config_vars。与 trans_var 一样，每个 <declaration> 可以是一条完整的类属性（变量）声明、typedef、约束或注释，例如：

```
config_var = // Configuration variable specified in interface template file
config_var = int count = 4;
```

（也可以在公共模板文件中使用 config_var，将声明添加到顶层配置类中。）

```
if_port     = <variable_declaration>
if_port     = <variable_declaration>
...
```

每项 if_port 都必须是一条适合包含在 SystemVerilog 接口中的完整声明，例如：

```
if_port     = parameter AW = 4;
if_port     = parameter DW = 4;
if_port     = logic          clock;
if_port     = logic          enable;
if_port     = logic [AW-1:0] addr;
if_port     = logic [DW-1:0] data;
```

这些声明会包含在与此 agent 组件关联的接口中。注意，可以引入参数，从而定义参数化接口，不过真正的参数化接口需要特殊处理。trans_vars 与 if_ports 之间通常存在某种关系，该关系由 UVM 驱动器和监视器组件的行为定义，但不必是一一对应关系。驱动器和监视器组件在驱动和采样接口变量时，会使用事务对象中的变量。

将 if_port 条目从模板文件复制到生成的代码时，会保留其格式，因此可以根据需要在模板文件中将变量名和 / 或初始值纵向对齐。

```
if_clock    = <name>
...
if_reset    = <name>
...
```

如果存在，if_clock 和 if_reset 分别用于标识接口的时钟变量和复位变量，这些变量也必须声明为 if_ports。注意，与 if_port 不同，这些设置必须指定变量名，而不是完整的变量声明。这两项设置唯一的实际作用，是生成赋值语句，分别将自动生成的时钟和复位生成代码中的信号赋给指定的时钟变量和复位变量。如果接口模板文件中设置了 generate_interface_instance = no（见下文），或公共模板文件中设置了 th_generate_clock_and_reset = no，这些设置就不起作用；在上述任一情况下，如果需要此类连接，用户都应自行将测试支架中的时钟和复位代码连接到接口中的时钟和复位变量。可以将 if_clock 和 if_reset 视为便利功能：在默认生成代码足够使用的简单情况下，自动将时钟和复位连接到接口。

```
generate_interface_instance = <yes or no>                                    [2016-04-18]
```

generate_interface_instance 可以设为 yes 或 no，取值不区分大小写。默认值为 yes，表示针对当前模板文件指定的接口生成一个 SystemVerilog 接口实例；如果 number_of_instances 大于 1，则生成多个接口实例。接口实例放置在生成的测试支架模块中。generate_interface_instance = no 允许用户自行实例化参数化接口，并覆盖默认参数值。

如果 generate_interface_instance = no，就不会根据当前模板文件生成接口实例；用户应使用 th_inc_inside_module 设置加入自己的接口实例化代码。当前接口模板文件中的所有 if_clock 或 if_reset 设置都会被忽略，因此用户应加入自己的代码，将所需的时钟和复位信号连接到自己实例化的接口。由于接口不是自动实例化的，相应配置对象中的虚接口不会被赋值。此时有两种选择：使用用户自定义代码片段对虚接口赋值（tb_inc_before_run_test 是适合的设置），或者不在 UVM 驱动器或监视器中访问这些虚接口（因为它们为 null）。

```
byo_interface  = <interface_name>                                            [2016-04-18]
```

如果存在，byo_interface 指定一个 BYO（Bring Your Own，即用户自备的）SystemVerilog 接口的名称。换句话说，生成的测试支架模块会实例化指定名称的接口，而不是自动生成的接口；该接口必须由用户提供。<interface_name> 不应为 <agent>_if，否则会与自动生成的接口名称冲突。可以使用 dut_source_path，将包含该接口的源文件作为 DUT 代码的一部分提供。byo_interface 设置既可与 generate_interface_instance = yes（默认值）配合使用，也可与 generate_interface_instance = no 配合使用。

同时使用 byo_interface 和 generate_interface_instance = yes（默认值）时，生成的接口实例名称与默认情况相同，即 <agent>_if_<suffix>。可以使用 if_clock 和 if_reset 设置，将自动生成的时钟和复位连接到接口实例。配置对象、驱动器和监视器中的虚接口会自动生成并设置，其类型改为 virtual <interface_name>，从而能够引用 BYO 接口实例。

同时使用 byo_interface 和 generate_interface_instance = no 时，接口实例名称由用户定义。if_clock 和 if_reset 设置会被忽略。配置对象、驱动器和监视器中的虚接口不会被自动赋值（因为接口实例名称未知），但可以在用户提供的代码片段中赋值（tb_inc_before_run_test 是适合的设置）。

参数化接口必须手动实例化（generate_interface_instance = no）。参数化接口可以是 BYO 接口，但并非必须如此。可以根据接口模板文件生成参数化接口。

```
uvm_seqr_class = <yes or no>
```

uvm_seqr_class 可以设为 yes 或 no，取值不区分大小写。默认值为 no，表示使用 typedef 定义该特定 agent 组件的序列仲裁器类型，例如：

```
typedef uvm_sequencer #(<trans_item>) <agent>_sequencer;
```

如果值为 yes，则生成一个显式的类，例如：

```
class <agent>_sequencer extends uvm_sequencer #(<trans_item>); ... endclass

agent_is_active  = <UVM_ACTIVE or UVM_PASSIVE>
```

agent_is_active 可以设为 UVM_ACTIVE 或 UVM_PASSIVE，通过相应地设置 agent 组件配置对象中的标志，决定 agent 组件以主动模式还是被动模式运行。默认值为 UVM_ACTIVE。配置对象在顶层环境 <top>_env 中创建，标志也在其中设置。

```
agent_checks_enable  = <yes or no>
```

agent_checks_enable 可以设为 yes 或 no，取值不区分大小写。默认值为 yes。此设置决定该 agent 组件配置对象中 checks_enable 标志的值，用户自定义代码片段应使用该标志启用或禁用检查。

```
agent_coverage_enable  = <yes or no>
```

agent_coverage_enable 可以设为 yes 或 no，取值不区分大小写。默认值为 yes。此设置决定该 agent 组件配置对象中 coverage_enable 标志的值，用户自定义代码片段应使用该标志启用或禁用覆盖率收集。如果设为 no，则不会实例化该 agent 组件的覆盖率订阅器组件。

```
agent_has_env    = <yes or no>
```

agent_has_env 可以设为 yes 或 no，取值不区分大小写。默认值为 no，表示 agent 组件直接实例化在顶层环境中（或另一个 agent 组件的环境中，参见 additional_agent）。如果值为 yes，agent 组件将实例化在自己的环境 <agent>_env 中，而该环境本身实例化在顶层环境 <top>_env 中。此标志除了决定验证环境的层次结构，还决定生成器所创建序列的结构（见下文）。agent_has_env

注意：与寄存器模型一起使用的 agent 组件始终具有自己的环境，因此对此类 agent 组件设置 agent_has_env = no 不起作用。

```
additional_agent = <agent>
additional_agent = <agent>
...
```

如果存在，每项 additional_agent 都定义一个将实例化在当前 agent 组件环境中的 agent 组件，前提是 agent_has_env = yes。如果 agent_has_env = no，指定附加 agent 组件将不起作用。agent 组件不得将自身指定为附加 agent 组件。

注意：在当前版本代码生成器的实现中，additional_agent 有许多限制，具体如下：

- 指定 additional_agent 的模板文件还必须指定 agent_has_env = yes。
- additional_agent 与 reg_access_name 互斥，即不应对具有寄存器模型的 agent 组件使用 additional_agent；这样做会导致相关附加 agent 组件不被实例化（除非它在自己的环境中实例化）。
- 作为附加 agent 组件出现的 agent 组件，自身不应设置 agent_has_env = yes（否则它会实例化两次，一次在自己的环境中，另一次作为其他环境中的附加 agent 组件）。
- 无论是在同一个接口模板文件中，还是在多个接口模板文件中，任一特定 agent 组件都不应作为 additional_agent 出现超过一次。
- 作为 additional_agent 出现的 agent 组件，自身必须设置 number_of_instances = 1（默认值）。

以下设置用于启用或禁止在相应类内部或紧接该类之后自动生成某些方法。所有设置的默认值均为 yes。禁止自动生成方法后，用户便可以自行定义这些方法。

```
trans_generate_methods_inside_class        = <yes or no>
trans_generate_methods_after_class         = <yes or no>
```

如果为 no，则禁止生成事务（序列项）类的 do_copy、do_compare、do_print、do_record 和 convert2string 方法。

```
agent_generate_methods_inside_class        = <yes or no>
agent_generate_methods_after_class         = <yes or no>
```

如果为 no，则禁止生成 agent 组件类的 build_phase 和 connect_phase 方法。

```
agent_env_generate_methods_inside_class    = <yes or no>
agent_env_generate_methods_after_class     = <yes or no>
```

如果为 no，则禁止生成 agent 组件环境类的 build_phase 和 connect_phase 方法（此处不是指顶层环境，后者可以通过公共模板文件控制）。

```
agent_config_generate_methods_inside_class = <yes or no>                    [2016-02-19]
agent_config_generate_methods_after_class  = <yes or no>
```

如果为 no，则禁止生成 agent 组件配置（uvm_object）类的 new 方法。如果为 no，必须使用 agent_config_inc_inside_class 和 / 或 agent_config_inc_after_class 设置，在包含文件中提供构造函数。

```
agent_cover_generate_methods_inside_class  = <yes or no>
agent_cover_generate_methods_after_class   = <yes or no>
```

如果为 no，则禁止生成 agent 组件覆盖率（订阅器）类的 new、write 和 report_phase 方法，以及覆盖组 m_cov。如果为 no，必须使用 agent_cover_inc_inside_class 和 / 或 agent_cover_inc_after_class 设置，在包含文件中提供构造函数。

```
reg_cover_generate_methods_inside_class    = <yes or no>
reg_cover_generate_methods_after_class     = <yes or no>
```

若设为 no，则不生成 agent 组件环境覆盖率（订阅者）类的 new、write 和 report_phase 方法以及覆盖组 m_cov。若设为 no，必须在 include 文件中提供构造函数。

```
adapter_generate_methods_inside_class      = <yes or no>                     [2016-02-19]
adapter_generate_methods_after_class       = <yes or no>
```

若设为 no，则不生成寄存器适配器（uvm_reg_adapter）类的 reg2bus 和 bus2reg 方法。若设为 no，必须在 include 文件中提供这些方法。

```
driver_inc                         = <filename> [inline]
monitor_inc                        = <filename> [inline]
agent_cover_inc                    = <filename> [inline]
reg_cover_inc                      = <filename> [inline]
```

driver_inc、monitor_inc、agent_cover_inc 和 reg_cover_inc 为初学者提供了一种便于入门的方法，因为它们要求用户提供的自定义代码量最少。下文“扩展生成的代码”一节将介绍这些设置。以下设置则提供更大的灵活性。

以下设置指定 include 文件的名称，用于在自动生成的代码中插入用户自定义代码。每项设置仅用于插入一些代码，不执行其他操作，因此这些设置可以与其他设置自由组合。默认情况下，每项设置生成一条 `include 指令。如果将 inline 作为第二个参数给出，则会直接内嵌复制用户自定义代码，而不使用 `include 指令来包含代码。无论是否指定 inline，被包含的文件都会复制到 <project>/tb/include 目录。嵌套 `include 指令所引用的任何文件都不会复制到 <project>/tb/include 目录。

```
if_inc_inside_interface          = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的接口中。

```
trans_inc_before_class           = <filename> [inline]
trans_inc_inside_class           = <filename> [inline]
trans_inc_after_class            = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的事务（序列项）类中。

```
agent_inc_before_class           = <filename> [inline]
agent_inc_inside_class           = <filename> [inline]
agent_inc_after_class            = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的 agent 组件类中。

```
agent_prepend_to_build_phase     = <filename> [inline]
agent_append_to_build_phase      = <filename> [inline]
agent_append_to_connect_phase    = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的 agent 组件类的 build_phase 或 connect_phase 方法中。要求 agent_generate_methods_after_class = yes（默认值）。

```
sequencer_inc_before_class       = <filename> [inline]
sequencer_inc_inside_class       = <filename> [inline]
sequencer_inc_after_class        = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的序列仲裁器类中。要求 uvm_seqr_class = yes。

```
driver_inc_before_class          = <filename> [inline]
driver_inc_inside_class          = <filename> [inline]
driver_inc_after_class           = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的驱动器类中。与 driver_inc 独立起作用。

```
monitor_inc_before_class         = <filename> [inline]
monitor_inc_inside_class         = <filename> [inline]
monitor_inc_after_class          = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的监视器类中。与 monitor_inc 独立起作用。

```
agent_cover_inc_before_class     = <filename> [inline]
agent_cover_inc_inside_class     = <filename> [inline]
agent_cover_inc_after_class      = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的 agent 组件覆盖率（订阅者）类中。与 agent_cover_inc 独立起作用。

```
agent_config_inc_before_class    = <filename> [inline]
agent_config_inc_inside_class    = <filename> [inline]
agent_config_inc_after_class     = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的 agent 组件配置类中（不是顶层配置类；顶层配置类可通过公共模板文件控制）。

```
agent_copy_config_vars           = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的顶层环境类的 build_phase 方法中，以便将任意用户自定义配置变量从顶层配置对象（参见公共模板文件中的 config_var）复制到 agent 组件配置对象（参见接口模板文件中的 config_var）。agent_copy_config_vars 主要用于 nested_config_objects = no（默认值）的情况。它也可用于 nested_config_objects = yes 的情况，不过通常没有必要，因为配置对象嵌套时，在配置对象之间复制变量的需求较少。

如果 number_of_instances 大于 1，指定文件只会被包含一次，其中可以包含对多个配置对象实例的赋值，这些配置对象实例对应于 agent 组件的多个实例。例如：

```
// m_config is a reference to the top-level configuration object
// m_agent_0_config and m_agent_1_config are references to two agent configuration objects
// There are two config_vars n_0 and n_1 in the top-level configuration object
// and one config_var n in the agent configuration object, of which there are two instances

m_agent_0_config.n = m_config.n_0;
m_agent_1_config.n = m_config.n_1;

agent_env_inc_before_class       = <filename> [inline]
agent_env_inc_inside_class       = <filename> [inline]
agent_env_inc_after_class        = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的 agent 组件环境类中（不是顶层环境类；顶层环境类可通过公共模板文件控制）。

```
agent_env_prepend_to_build_phase  = <filename> [inline]
agent_env_append_to_build_phase   = <filename> [inline]
agent_env_append_to_connect_phase = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的 agent 组件环境类的 build_phase 或 connect_phase 方法中。要求 agent_env_generate_methods_after_class = yes（默认值）。

```
reg_cover_inc_before_class       = <filename> [inline]
reg_cover_inc_inside_class       = <filename> [inline]
reg_cover_inc_after_class        = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的 agent 组件环境覆盖率（订阅者）类中。

```
adapter_inc_before_class         = <filename> [inline]                    [2016-02-19]
adapter_inc_inside_class         = <filename> [inline]
adapter_inc_after_class          = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的寄存器适配器类中。

```
agent_seq_inc                    = <filename> [inline]
```

将指定文件中的用户自定义代码包含到文件 <agent>_seq_lib.sv 的末尾，该文件包含 <agent>_default_seq。

```
agent_env_seq_inc                = <filename> [inline]
reg_seq_inc                      = <filename> [inline]
```

将指定文件中的用户自定义代码包含到文件 <agent>_env_seq_lib.sv 的末尾，该文件包含 <agent>_env_default_seq。（reg_seq_inc 是 agent_env_seq_inc 的别名，为保持向后兼容而保留。）

```
agent_factory_set = <original-classname>  <replacement-classname>
```

在 <top>_test 类的 build_phase 方法中生成对 set_type_override 的调用。可与 agent_seq_inc 或 agent_env_seq_inc 配合使用，将默认序列替换为用户自定义序列。top_factory_set 和 agent_factory_set 的行为完全相同，因为两者都按类型进行工厂覆盖。下文“添加用户自定义序列”和“

添加用户自定义寄存器序列”两节将更详细地介绍 agent_seq_inc、agent_env_seq_inc、req_seq_inc 和 agent_factory_set。

```
reg_access_name = <agent>  <mode>                          [Deprecated in 2015-06-29]
```

<agent> 必须是 agent 组件的名称，与 agent_name 给出的名称相同。<mode> 必须为 WR（写/读）、WO（只写）或 RO（只读）。reg_access_name 已弃用，并已由 reg_access_mode 取代。

```
reg_access_mode = <mode>
```

指定该 agent 组件将连接到寄存器模型，并且用户自定义寄存器序列可以按给定模式访问寄存器。<mode> 可以是 WR（写/读）、WO（只写）或 RO（只读）。<mode> 的实际取值与 uvm_reg_adapter 内部生成的代码有关。（取代 reg_access_name。）

```
reg_access_map = <map>
```

指定表示根地址映射的变量名，该根地址映射用于通过此 agent 组件访问寄存器。无论寄存器本身在寄存器模型中的哪个位置，每个用于寄存器访问的 agent 组件都必须连接到寄存器模型根（顶层）uvm_reg_block 类中的一个独立根映射。生成的代码会将此映射连接到相应的适配器、预测器和序列仲裁器。默认值为 <agent>_map（用于保持与代码生成器早期版本的向后兼容）。如果 number_of_instances 大于 1，则通过添加后缀 _0、_1、_2、... 来区分映射变量名，不过后续版本将以不同方式处理。

```
reg_access_block_instance = <instance>
```

指定寄存器模型中 uvm_reg_block 实例的对象路径；从寄存器序列访问寄存器时，将以该实例为根，即与该 agent 组件关联的生成的 配置（config）、环境（env）、覆盖率（coverage）和序列（sequence）类中的 regmodel 变量所引用的寄存器块。此值只影响序列如何访问寄存器模型中的寄存器；它与 reg_access_map 的值无关（reg_access_map 始终位于顶层寄存器块中）。将 reg_access_block_instance 设置为顶层寄存器模型以外的实例，可以使寄存器序列访问寄存器时不依赖于寄存器在寄存器模型中的层次位置。对象路径可以是层次路径，开头的点号（句点）字符可写可不写。默认值为 <agent>（用于保持与代码生成器早期版本的向后兼容）。如果 number_of_instances 大于 1，则通过添加后缀 _0、_1、_2、... 来区分实例变量名，不过后续版本将以不同方式处理。

例如：

```
reg_access_block_instance = a.b.c
```

从寄存器序列发起的寄存器访问将采用以下形式：

```
regmodel.<register_name>.write(...);
regmodel.<register_name>.read(...);
```

其中 regmodel 是对实例 <top-level-regmodel>.a.b.c 的引用。也可以按如下方式将实例名设为 null：

```
reg_access_block_instance =
```

在这种情况下，从寄存器序列发起的任何寄存器访问都需要使用从顶层寄存器模型开始的绝对引用：

```
regmodel.a.b.c.<register_name>.write(...);
regmodel.a.b.c.<register_name>.read(...);

reg_access_block_type = <type>
```

指定 reg_access_block_instance 所选定的 uvm_reg_block 类的类型。代码生成器使用此类型作为生成的配置（config）、环境（env）、覆盖率（coverage）和序列（sequence）类中局部 regmodel 变量的类型。如果 reg_access_block_instance 为 null，reg_access_block_type 必须指定顶层寄存器块的类型。如

果省略 reg_access_block_instance，reg_access_block_type 必须指定顶层寄存器块中名为 <agent> 的变量的类型（用于保持与代码生成器早期版本的向后兼容）。

```
uvm_reg_kind = <kind>
uvm_reg_addr = <addr>
uvm_reg_data = <data>
```

<kind> 必须是表示访问类型的 trans_var 的名称（0 = 读，1 = 写）。<addr> 必须是表示寄存器读/写地址的 trans_var 的名称。<data> 必须是表示寄存器读/写数据的 trans_var 的名称。

参见紧接下文的“寄存器模板文件”一节。

05 寄存器模板文件

原件页码：17-19

寄存器模板文件已弃用。请勿使用寄存器模板文件！此文件提供的功能可改为通过接口模板文件和公共模板文件中的设置来实现。

```
perl easier_uvm_gen.pl if1.tpl if2.tpl if3.tpl reg.tpl
```

寄存器模型本身必须放在 regmodel_file 指定的文件中（默认值为 regmodel.sv），该文件位于运行脚本的目录中。脚本不会生成寄存器模型：必须单独生成（或手工编写）。

reg.tpl 文件用于标识寄存器模型中寄存器块的名称和结构。顶层寄存器块将在顶层环境中实例化，每个较低层级的寄存器块将在其地址映射所连接的 agent 组件的 环境中实例化。reg.tpl 必须采用以下格式：

```
regmodel_name      = <top-level-register-block-classname>
regmodel_file      = <register-model-filename>
regmodel_sub_block = <register-sub-block-classname>  <instance/agent-name>
regmodel_sub_block = <register-sub-block-classname>  <instance/agent-name>
...

In [2015-06-29] these settings were replaced as follows:
regmodel_name        replaced by top_reg_block_type in the common template file
regmodel_file        replaced by regmodel_file in the common template file
regmodel_sub_block   replaced by reg_access_block_type in the interface template file
```

<top-level-register-block-classname> 必须是寄存器模型中顶层 uvm_reg_block 的类名。<register-model-filename> 必须是包含寄存器模型的文件的名称，相对于运行脚本的目录指定。如果省略此参数，文件名默认为 regmodel.sv。<register-sub-block-classname> 必须是寄存器模型中与某个特定 agent 组件关联的 uvm_reg_block 的类名。<instance/agent-name> 必须是顶层寄存器块类中该特定子块的变量名。此外，该变量名必须是与该寄存器子块关联的 agent 组件的名称。

除了寄存器模板文件之外，每个 agent 组件的接口模板文件还必须指定事务的特定变量与通用寄存器访问所需的访问类型、地址和数据之间的映射。

如果 agent 组件与寄存器块关联，则 agent_has_env 不得设为 no。换言之，与 agent 组件关联的寄存器块必须在该特定 agent 组件专用的环境中实例化。可以混合使用带寄存器块的 agent 组件和不带寄存器块的 agent 组件，只要每个带寄存器块的 agent 组件都有自己的环境，其余 agent 组件的 agent_has_env = yes 或 no 均不受限制。

06 公共模板文件

原件页码：19-26

除了命令行中列出的接口模板文件之外，还必须有且仅有一个公共模板文件，其中包含所有不涉及特定 agent 组件/接口的代码生成器设置。公共模板文件默认名为 common.tpl，但也可以使用 -m 命令行选项选择其他文件名。

有一个必填设置 dut_top：

```
dut_top  = <name>
```

dut_top 必须是 DUT 顶层模块的 SystemVerilog 模块名（不是实例名）。

其余设置均为可选：

```
dut_source_path = <path>
```

dut_source_path 指定包含 DUT 文件的目录路径。该路径必须相对于运行脚本的目录指定。脚本会递归地将 <dut_source_path> 目录的内容复制到 <project>/dut 目录，以便脚本处理 DUT 文件的副本，而非原文件。此目录可以包含一个名为 files.f 的文件；如果该文件存在，则必须包含编译 DUT 所需的所有 SystemVerilog 源文件名和命令行选项，但不应引用 UVM 基类库本身的文件（因为这些文件会被包含在自动生成的编译脚本中）。如果 files.f 不存在，生成器将创建一个 files.f，按字母顺序列出 DUT 目录中所有扩展名为 .sv 的文件。如果源文件需要以字母顺序以外的顺序编译，或者有源文件不直接位于 DUT 目录内，则用户必须提供 files.f。如果未指定 dut_source_path，默认值为 dut。

```
inc_path  = <path>
```

inc_path 指定一个目录的路径，该目录可以包含存放用户自定义代码片段的文件，这些代码片段将被包含到生成的代码中。（此功能在 2015-06 版本中得到大幅增强。）该路径必须相对于运行脚本的目录指定。脚本会将 <inc_path> 目录中的文件复制到 <project>/tb/include 目录，以便处理 include 文件的副本，而非原文件。嵌套 `include 指令所引用的任何文件都不会复制到 <project>/tb/include 目录。如果存在 include 文件，必须在接口模板文件和公共模板文件中使用适当的模板设置（如 driver_inc_inside_class）指定它们的文件名。如果未指定 inc_path，默认值为 include。

inc_path 目录中的 include 文件可以组织到子目录中。相应的设置此时应包含相对路径名，例如：

```
driver_inc  = bus1/bus1_do_drive.sv  inline
monitor_inc = bus1/bus1_do_mon.sv    inline

dut_pfile = <path>
```

dut_pfile 指定 DUT 引脚列表文件的路径（见下文）。默认值为 pinlist。

```
prefix    = <prefix>
```

若提供 prefix，则指定构造与顶层验证环境关联的文件、模块、包和 UVM 类的名称时所用的前缀。此处在公共模板文件中指定的值可使用命令行上的 -p 选项覆盖。默认值为 top。

```
project   = <project>
```

若提供 project，则指定 <project> 目录的名称；该目录将在当前目录下创建，并且必须以简单目录名的形式给出，不能给出路径。脚本生成的所有文件均包含在 <project> 目录中或其子目录中。默认值为 generated_tb。

```
dut_iname = <instance_name>
```

若提供 dut_iname，则指定在测试平台模块中生成的 DUT 模块实例的实例名，该实例将位于文件 <project>/tb/<top>_tb/sv/<top>_tb.sv 中。默认值为 uut。

```
copyright = <text>
name      = <text>
email     = <text>
tel       = <text>
dept      = <text>
company   = <text>
year      = <text>
version   = <text>
```

若提供 copyright 文本、作者的 name、email、tel、dept 和 company，以及 year 和 version，则这些设置指定的文本字段将插入每个文件顶部自动生成的文件头中。除其中自动生成的文件名和描述之外，各文件的文件头完全相同。

```
generate_file_header = <yes or no>                                    [2016-02-15]
```

若设为 no，则不生成包含上述文本字段的自动生成文件头。

```
file_header_inc = <filename>                                          [2016-02-15]
```

将指定文件中的用户自定义样板代码包含到每个文件的顶部，紧接在自动生成的文件头之前。文件头文本始终以内嵌方式插入（不使用 `include）。可以仅使用自动生成的文件头、仅使用用户自定义文件头、两者都不用，或两者同时使用。如果希望各文件的文件头文本有较大差异，请改用 * _inc_before_class 设置。

```
timeunit        = <integer><unit>
timeprecision   = <integer><unit>
```

timeunit 和 timeprecision 指定要包含在生成的 SystemVerilog 模块中的 SystemVerilog timeunit 和 timeprecision 声明的值。如果均未指定，默认值为 1ns/1ps。

```
backup          = <yes or no>
```

backup 指定在用同名的新项目目录覆盖任何现有项目目录之前，是否对现有项目目录进行备份。备份目录的文件名扩展名为 .bak。默认值为 yes。

```
comments_at_include_locations = <yes or no>
```

指定是否生成以下注释：标明可以使用模板文件设置来插入用户自定义代码的位置的注释，以及标明可以禁止生成自动代码的位置的注释。默认值为 yes。

```
common_pkg      = <filename>
common_env_pkg  = <filename>
```

若提供 common_pkg，则指定一个必须包含包声明的文件的名称。该文件必须位于 dut_source_path 目录中。文件名无需与包名相同。公共包对 DUT 和整个 UVM 环境可见，这意味着公共包将最先编译，并且会在生成的代码中加入包导入声明。

若提供 common_env_pkg，则指定一个必须包含包声明的文件的名称。该文件必须位于 inc_path 目录中。文件名无需与包名相同。公共环境包对整个 UVM 环境可见，这意味着公共环境包将最先编译，并且会在生成的代码中加入包导入声明，但对 DUT 不可见。

如果提供这两个包，它们将是自动生成的代码中每个包最先导入的两个包。因此，在没有前向 typedef 声明的情况下，它们不能包含对自动生成的类的引用。只能有一个公共包和一个公共环境包。

```
regmodel_file   = <filename>
```

若提供 regmodel_file，则指定包含寄存器模型的文件的名称，相对于运行脚本的目录指定。如果省略此参数，文件名默认为 regmodel.sv。（此设置的作用与 reg.tpl 中的 regmodel_file 相同。）

```
top_reg_block_type = <classname>
```

若提供 top_reg_block_type，则指定寄存器模型中顶层 uvm_reg_block 类的类型。代码生成器将其用作顶层环境和顶层虚拟序列中顶层 regmodel 变量的类型。如果需要寄存器模型，此设置为必填项。（此设置的作用与 reg.tpl 中的 regmodel_name 相同。）

```
nested_config_objects = <yes or no>                                    [2016-02-19]
```

指定是否生成嵌套的配置对象，即在顶层配置对象中包含对 agent 组件配置对象的引用。为保持与代码生成器早期版本的向后兼容，默认值为 no。

若设为 yes（推荐），则顶层配置对象包含对 agent 组件配置对象的引用。顶层配置对象在顶层模块中实例化，agent 组件配置对象则在顶层配置对象的构造函数（new）中实例化。这种方法的优点是：不必在顶层配置对象和 agent 组件配置对象之间重复存储或复制信息；顶层配置对象可以包含引用 agent 组件配置对象内部变量的约束；并且除了可以从顶层环境类对任意配置对象中的变量赋值外，还可以从顶层模块或测试类对其赋值。默认情况下，生成的顶层配置对象仅包含对 agent 组件配置对象的引用。

若设为 no（默认值），则顶层配置对象与 agent 组件配置对象彼此独立。这种方法的特点是，agent 组件配置对象完全封装在顶层环境类内部，而顶层模块和测试类只能看到顶层配置对象。顶层配置对象在顶层模块中实例化，agent 组件配置对象则在顶层环境类中实例化。顶层环境类负责根据来自顶层配置对象的信息创建并初始化 agent 组件配置对象。默认在每个 agent 组件配置类中生成的变量也会在顶层配置类中重复声明，并在顶层环境类的 build_phase 方法中从顶层配置对象复制到 agent 组件配置对象。

```
config_var      = <declaration>
config_var      = <declaration>
...
```

每条 config_var 声明都会包含到生成的顶层配置类中。每个 <declaration> 可以是完整的类属性（变量）声明、typedef、约束或注释。config_var 也可以用在接口模板文件中，为各个 agent 组件配置类添加声明。参见 agent_copy_config_vars。

以下设置用于启用或禁止在相应类或模块内部、或紧接其后自动生成某些代码。每项设置的默认值均为 yes。禁止自动生成方法后，便可由用户自定义这些方法。

```
th_generate_clock_and_reset             = <yes or no>
```

若设为 no，则不生成测试支架模块 <project>/tb/<top>_tb/sv/<top>_th.sv 中的时钟和复位声明及赋值。这些声明和赋值依赖于接口模板文件中的 if_clock 和 if_reset 设置。如果存在多个时钟，则生成的各个时钟将具有相同的周期和占空比，复位信号的情况也类似。

```
top_env_generate_methods_inside_class    = <yes or no>
top_env_generate_methods_after_class     = <yes or no>
```

若设为 no，则不生成顶层环境类的 build_phase、connect_phase 和 run_phase 方法。

```
top_env_generate_run_phase              = <yes or no>                  [2016-02-15]
```

若设为 no，则仅禁止生成顶层环境类的 run_phase 方法。除非同时指定 top_env_generate_methods_inside/after_class，否则仍会生成 build_phase 和 connect_phase。

```
top_env_generate_end_of_elaboration     = <yes or no>
```

若设为 no，则不生成顶层环境类的 end_of_elaboration_phase 方法。生成的方法会打印消息详细程度阈值、拓扑结构和工厂覆盖，以帮助调试验证环境。

```
top_env_config_generate_methods_inside_class = <yes or no>             [2016-02-19]
top_env_config_generate_methods_after_class  = <yes or no>
```

若设为 no，则不生成顶层配置（uvm_object）类的 new 方法。若设为 no，必须使用 top_env_config_inc_inside_class 和/或 top_env_config_inc_after_class 设置，在 include 文件中提供构造函数。

```
test_generate_methods_inside_class       = <yes or no>
test_generate_methods_after_class        = <yes or no>
```

若设为 no，则不生成测试类的 build_phase 方法。

以下设置指定 include 文件的名称，用于在自动生成的代码中插入用户自定义代码。每项设置仅用于插入一些代码，不执行其他操作，因此这些设置可以与其他设置自由组合。默认情况下，每项设置生成一条 `include 指令。如果将 inline 作为第二个参数给出，则会直接内嵌复制用户自定义代码，而不使用 `include 指令来包含代码。无论是否指定 inline，被包含的文件都会复制到 <project>/tb/include 目录。嵌套 `include 指令所引用的任何文件都不会复制到 <project>/tb/include 目录。

```
th_inc_inside_module             = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的测试支架模块 <project>/tb/<top>_tb/sv/<top>_th.sv 中，位于 DUT 实例化之前。

```
tb_inc_inside_module             = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的顶层测试平台模块 <project>/tb/<top>_tb/sv/<top>_tb.sv 中。

```
tb_prepend_to_initial            = <filename> [inline]                    [2016-04-18]
```

将指定文件中的用户自定义代码包含到生成的顶层测试平台模块 <project>/tb/<top>_tb/sv/<top>_tb.sv 中调用 run_test() 的生成的 initial 块内部的最前面。

```
tb_inc_before_run_test           = <filename> [inline]                    [2016-04-18]
```

将指定文件中的用户自定义代码包含到顶层测试平台模块 <project>/tb/<top>_tb/sv/<top>_tb.sv 中生成的 initial 块内部，紧接在调用 run_test() 之前。tb_inc_before_run_test 可用于从顶层模块对配置对象中的变量赋值，例如设置虚拟接口，使其引用用户自定义的接口实例（参见 generate_interface_instance）。除了虚拟接口之外，配置对象中的变量通常最好从测试中设置（参见 test_prepend_to_build_phase）。

```
tb_generate_run_test             = <yes or no>                            [2016-02-19]
```

若设为 no，则不生成顶层测试平台模块 <project>/tb/<top>_tb/sv/<top>_tb.sv 中调用 run_test() 的 initial 块。随后可以使用 tb_inc_inside_module 提供自己的顶层配置对象和对 run_test() 的调用。

```
test_inc_before_class            = <filename> [inline]
test_inc_inside_class            = <filename> [inline]
test_inc_after_class             = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的测试类中。

```
test_prepend_to_build_phase      = <filename> [inline]
test_append_to_build_phase       = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的测试类的 `build_phase` 方法中。要求 `test_generate_methods_after_class = yes`（这是默认值）。

```
top_env_inc_before_class         = <filename> [inline]
top_env_inc_inside_class         = <filename> [inline]
top_env_inc_after_class          = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的顶层环境类中。

```
top_env_prepend_to_build_phase   = <filename> [inline]
top_env_append_to_build_phase    = <filename> [inline]
top_env_append_to_connect_phase  = <filename> [inline]
top_env_append_to_run_phase      = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的顶层环境类的 `build_phase`、`connect_phase` 或 `run_phase` 方法中。要求 `top_env_generate_methods_after_class = yes`（这是默认值）。

```
top_env_config_inc_before_class  = <filename> [inline]
top_env_config_inc_inside_class  = <filename> [inline]
top_env_config_inc_after_class   = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的顶层配置类中（不是 agent 组件的配置类，后者可以通过接口模板文件进行控制）。

```
top_env_config_append_to_new     = <filename> [inline]
```

将指定文件中的用户自定义代码包含到生成的顶层配置类的 `new` 方法中。例如，插入的代码可用于初始化顶层配置类中使用 `config_var` 指定的变量。

```
top_seq_inc               = <filename> [inline]
```

将指定文件中的用户自定义代码包含到文件 `<top>_seq_lib.sv` 的末尾，该文件包含 `<top>_default_seq`。

```
top_factory_set           = <original-classname>  <replacement-classname>
```

在 `<top>_test` 类的 `build_phase` 方法中生成对 `set_type_override` 的调用。可以与 `top_seq_inc` 配合使用，以用户自定义序列替换默认序列。`top_factory_set` 和 `agent_factory_set` 的行为完全相同，因为两者都按类型执行工厂覆盖。

```
top_default_seq_count     = <integer>                                    [2015-11]
```

指定顶层默认虚拟序列中使用的重复次数，用于决定启动较低层序列的次数。这是一个便捷设置，仅当实际使用顶层默认虚拟序列来演示生成的代码或对其进行流程连通性检查时才有意义。如果使用工厂覆盖将顶层默认虚拟序列替换为用户自定义序列，此设置就没有作用。默认值为 `1`。

```
uvm_cmdline               = <flag> <flag> <flag> ...                     [2015-11]
```

指定一个标志列表，这些标志将作为仿真器命令行参数传递给 UVM 命令行处理器。多个标志应以空格分隔。

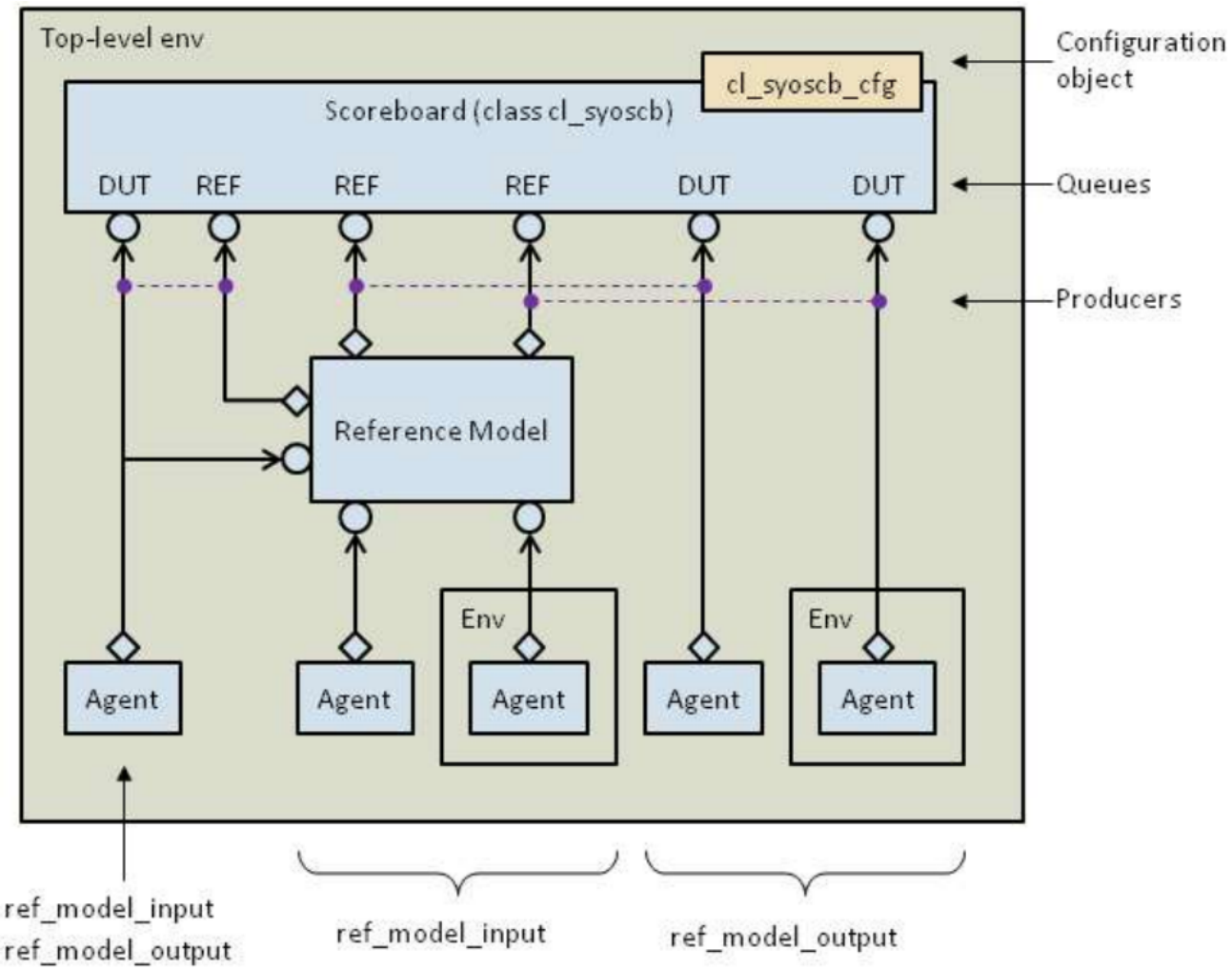
示例

```
uvm_cmdline = +UVM_VERBOSITY=UVM_HIGH +UVM_OBJECTION_TRACE
```

07 通用模板文件 - 参考模型和记分板

原件页码：26-29

代码生成器可以创建一个或多个参考模型，每个参考模型都关联一个独立的 Syosil UVM Scoreboard 实例。每个参考模型从某些 agent 组件接收事务，并向其记分板发送事务。每个记分板从其参考模型接收表示 DUT 预期行为的事务，同时从某些 agent 组件接收表示 DUT 实际行为的事务。Syosil UVM Scoreboard 自身的源代码必须从 www.syosil.com 单独下载。更多详细信息请参阅 Syosil UVM Scoreboard 随附的文档。



图内术语：Top-level env = 顶层环境；Scoreboard = 记分板；Configuration object = 配置对象；Queues = 队列；Producers = 产生源；Reference Model = 参考模型。DUT 队列保存实际响应，REF 队列保存预期响应；ref_model_input/ref_model_output 是模板设置名。

图：参考模型和记分板

记分板预定义的比较方法比较来自同一生产者的两个事务：一个来自 `"DUT"` 队列、表示实际响应的事务，以及一个来自 `"REF"` 队列、表示预期响应的事务。

参考模型和记分板由通用模板文件中的以下设置控制：

```
syosil_scoreboard_src_path = <path>                                    [2015-11]
```

`syosil_scoreboard_src_path` 指定包含 Syosil UVM Scoreboard 源文件的目录路径。此路径必须相对于运行脚本的目录指定。在可从 www.syosil.com 下载的发行文件中，此目录名为 `./src`。Easier UVM 代码

生成器已使用 Syosil UVM Scoreboard 的 `1.0.2.5` 版本进行过测试，并且有望适用于该版本及后续版本。当且仅当使用以下设置实例化参考模型和记分板时，才需要此设置。

```
ref_model_input           = <ref_model> <agent_instance_name>              [2015-11]
```

`ref_model_input` 指定一个向参考模型的某个输入（技术上是一个 `uvm_analysis_imp`）发送事务的 agent 组件。参考模型可以有任意数量的输入。`<ref_model>` 是标识特定参考模型的标签，并在生成的代码中用作顶层环境内参考模型、记分板及相关组件实例名称的词根。可以有任意数量的参考模型。每个参考模型都会关联一个 Syosil UVM Scoreboard。通用模板文件中所有涉及某个特定参考模型及其关联记分板的设置，都必须使用相同的标签。`<agent_instance_name>` 必须是一个 SystemVerilog 变量的名称，该变量持有对某个 UVM agent 对象的引用。它可以是分层对象引用，也就是说，可以包含点号，以便引用实例化在其自身环境中的 agent 组件（使用设置 `agent_has_env = yes`）。在生成的代码中，指定 agent 组件的分析端口将连接到参考模型的一个分析导出（类型为 `uvm_analysis_imp`）。

```
ref_model_output          = <ref_model> <agent_instance_name>              [2015-11]
```

`ref_model_output` 指定一个向记分板发送分析事务的 agent 组件，这些事务反映 DUT 的实际输出。参考模型可以有任意数量的输出。记分板将来自指定 agent 组件的事务（Syosil UVM Scoreboard 内的 `"DUT"` 队列）与参考模型输出的预期事务（`"REF"` 队列）进行比较。换言之，指定的 agent 实例实际上并不是参考模型的输出，而是代表 DUT 的一个输出，由记分板将其与参考模型进行比较。在生成的代码中，指定 agent 组件的分析端口将连接到记分板的一个分析导出，而参考模型中相应的分析端口将连接到记分板的第二个分析导出。

对于同一个参考模型，同一个 `<agent_instance_name>` 可以同时用作 `ref_model_input` 和 `ref_model_output`。在这种情况下，构成参考模型某个输入的分析事务流也会直接传递给记分板，在那里表示 DUT 的实际输出（`"DUT"` 队列）。参考模型将创建自己的分析事务流，该事务流也会传递给记分板以进行比较（`"REF"` 队列）。

```
ref_model_inc_before_class = <ref_model> <filename> [inline]               [2015-11]
ref_model_inc_inside_class = <ref_model> <filename> [inline]               [2015-11]
ref_model_inc_after_class  = <ref_model> <filename> [inline]               [2015-11]
```

将指定文件中的用户自定义代码包含到由指定 `<ref_model>` 标签标识的参考模型中。`ref_model_inc_after_class` 必须包含一个文件，该文件定义与作为参考模型一部分自动生成的 `uvm_analysis_imp` 导出相关联的 `extern function void write...` 方法，每个导出对应一个 `write` 方法。例如：

```
function void <ref_model>::write_<ref_model>_0(<tx_type> t);
  ...
endfunction
```

请注意，参考模型的分析导出和分析端口都使用从 `0` 开始递增的 `_N` 后缀编号。理解这一点最简单的方法，是在尝试编写自己的用户自定义代码片段之前，先查看为参考模型生成的代码。

```
ref_model_compare_method   = <ref_model> <io or iop or ooo>                [2015-11]
```

`ref_model_compare_method` 指定由给定记分板实例使用的比较方法。Syosil UVM Scoreboard 提供三种预定义比较方法：按顺序（In Order，`io`）、按生产者分别保序（In Order by Producer，`iop`）和乱序（Out of Order，`ooo`）；也可以定义自己的比较方法。不同的记分板实例可以使用不同的比较方法。比较方法通过生成代码中的工厂覆盖来设置。如果以后添加了新的比较方法，只要新的比较类遵循现有三个比较类的命名约定（例如 `cl_syoscb_compare_ooo`），就应该能够通过调整此设置的值来选择新的比较方法。

08 通用模板文件 - 支持加速/硬件仿真的环境

原件页码：29-30

`dual_top`、`split_transactors` 和 `agent_inc_inside_bfm` 这三个设置与生成支持加速/硬件仿真的环境有关。在这种环境中，实例化 DUT 的“HDL 域”可以综合到加速器或硬件仿真设备上，而实例化 UVM 环境的“HVL 域”则严格不含时序，也就是说，不包含任何延迟或时钟。

```
dual_top = <yes or no>                                          [2016-01-21]
```

`dual_top` 创建两个顶层模块，而不是在顶层测试平台模块之下实例化测试支架模块。默认值为 `no`。

```
split_transactors = <yes or no>                                 [2016-01-21]
```

`split_transactors` 要求将每个事务处理器（驱动器和监视器）拆分为跨越 HVL 域和 HDL 域的分离式事务处理器。默认值为 `no`。`split_transactors` 执行以下操作：

- 强制设置 `dual_top = yes`。
- 在测试支架模块中为每个 agent 组件额外实例化一个接口。此接口旨在用作可综合的 BFM（总线功能模型），以驱动和监视 DUT 的引脚。
- 将 UVM 环境中的虚拟接口设置为指向这些 BFM，而非引脚级接口。

```
agent_inc_inside_bfm = <filename> [inline]                      [2016-01-21]
```

将指定文件中的用户自定义代码包含到生成的 BFM 中，该 BFM 是一个 SystemVerilog 接口。

要了解如何使用 `split_transactors` 创建支持加速的环境，请参阅教程第 5 部分。

09 引脚列表文件

原件页码：30-31

引脚列表文件（由通用模板文件中的 `dut_pfile` 指定）是包含 DUT 引脚列表的文本文件，生成器使用该列表来确定 DUT 模块实例的端口与测试平台变量之间的连接。该文本文件本身采用以下格式：

```
DEC = <variable_declaration>
DEC = <variable_declaration>
...
PAR = <parameter_name> <parameter_value>
PAR = <parameter_name> <parameter_value>
...
!<interface_name>
<dut_port_name> <variable_name>
<dut_port_name> <variable_name>
...
!<interface_name>
// Connections for a specific interface
<dut_port_name> <variable_name>
<dut_port_name> <variable_name>
...
!
// Ports connected to local variables
<dut_port_name> <variable_name>
<dut_port_name> <variable_name>
...
```

`<variable_declaration>` 必须是完整的变量声明，包括分号，例如 `logic [7:0] data;`。
`<parameter_name>` 是 DUT 模块实例化时某个 SystemVerilog 参数的名称。`<parameter_value>` 是 DUT 模块实例化时某个 SystemVerilog 参数的默认值。`<interface_name>` 可以是 SystemVerilog 接口

的名称，也可以是接口实例名称（见下文）。`<dut_port_name>` 必须是 DUT 上某个端口的名称。`<variable_name>` 必须是 SystemVerilog 接口中将要连接到该 DUT 端口的相应变量的名称。

`DEC` 用于指定局部变量声明，这些声明会包含在生成的代码中，位于 DUT 模块实例化之前，并且可以在 `!` 或 `!none` 后面的列表中连接到 DUT 的端口。换言之，`!` 或 `!none` 用于指定 DUT 端口与局部声明变量之间的连接，而不是 DUT 端口与接口中声明变量之间的连接。

`PAR` 用于指定生成的模块实例化语句的参数。每个参数都必须给出一个默认值。

列表中任意位置的 SystemVerilog 单行注释（`//`，不包括 `/* */`）都会原样复制到生成的代码中，例如可用来说明一组端口连接与某个特定接口相关联。任何以 `#` 字符开头的行都被视为普通注释，并会被忽略。

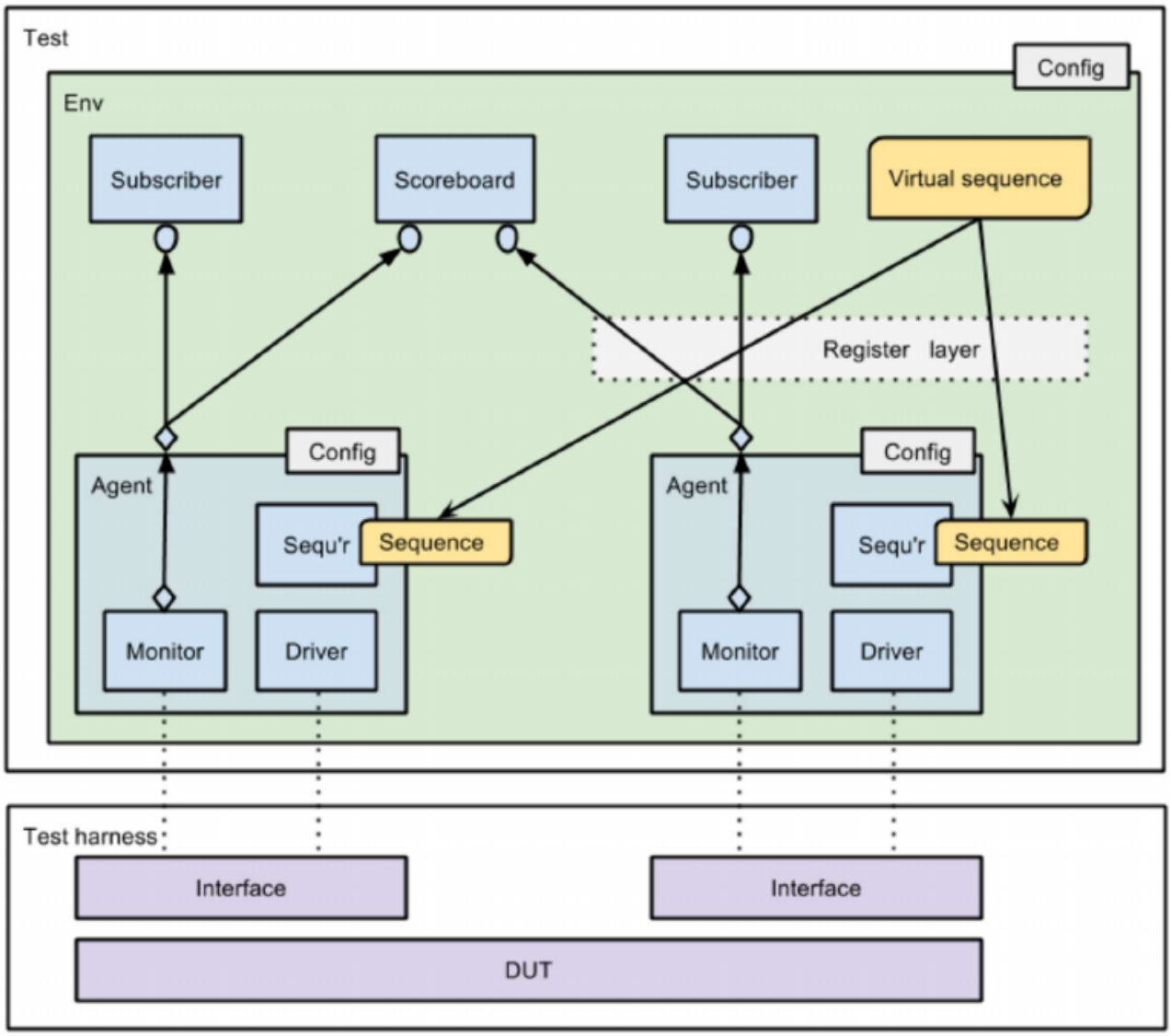
接口名称通常采用 `<agent>_if` 的形式。如果 `number_of_instances` 大于 `1`，可以在引脚列表文件的接口名称后添加后缀 `_0`、`_1`、`_2`、……，以区分同一接口的多个实例。如果 `number_of_instances = 1`，则引脚列表文件中的 `_0` 后缀是可选的（不过它会出现在生成的代码中）。

如果 `generate_interface_instance = no`，接口名称应为用户自定义的接口实例名称（不受 `number_of_instances` 影响）。[2016-04-18]

10 生成的结构

原件页码：31-35

生成的模块、接口和类的集合可以用下图表示。每个接口模板文件都对应一个接口以及与某个 agent 组件相关的所有类，包括一个配置对象和一个序列。如果 `agent_has_env = yes` 或该 agent 组件具有寄存器模型，则每个 agent 组件都会在其自身的环境中实例化，而且该环境会有其自身关联的虚拟序列（见下文）。



图内术语：Test = 测试；Env = 环境；Config = 配置；Subscriber = 订阅器；Scoreboard = 记分板；Virtual sequence = 虚拟序列；Register layer = 寄存器层；Sequence = 序列；Seqr = 序列仲裁器；Monitor = 监视器；Driver = 驱动器；Test harness = 测试支架；Interface = 接口。

图：生成的各类“对象”

生成的文件结构如下：

```
<project>/dut                                        Copy of DUT directory

<project>/tb/<agent>/sv/<agent>_<trans>.sv           Per-interface/agent
                        <agent>_if.sv
                        <agent>_config.sv
                        <agent>_driver.sv
                        <agent>_monitor.sv
                        <agent>_sequencer.sv
                        <agent>_agent.sv
                        <agent>_coverage.sv
                        <agent>_seq_lib.sv           Sequences
                        <agent>_package.sv           Package

                        <agent>_env.sv               If agent_has_env = yes
                        <agent>_env_seq_lib.sv       If agent_has_env = yes

<project>/tb/<agent>_env/sv/reg2<agent>_adapter.sv   If agent uses a regmodel
                            <agent>_env_coverage.sv  If agent uses a regmodel
                            <agent>_env.sv           If agent uses a regmodel
                            <agent>_env_seq_lib.sv   If agent uses a regmodel
                            <agent>_env_package.sv   If agent uses a regmodel

<project>/tb/<top>/sv/<top>_config.sv                Top-level env
                       <top>_env.sv
                       <top>_seq_lib.sv
                       <top>_package.sv

<project>/tb/<top>_test/sv/<top>_test.sv             Test
                           <top>_test_package.sv

<project>/tb/<top>_tb/sv/<top>_tb.sv                 Modules
                         <top>_th.sv

<project>/tb/include                                 Copy of user-defined include files

<project>/sim                                        Simulation scripts
```

给定两个接口模板，没有寄存器模型，并且每个 agent 组件均设置 `agent_has_env = no` 时，生成代码的结构如下：

```
testbench (module)
 ↳ test harness (module)
   ↳ interface1 (interface instance)
     interface2 (interface instance)
     DUT (module instance)

 ↳ top_config (object)
   test (object)
      top_env
       ↳ config1
         agent1
          ↳ sequencer1
            driver1
            monitor1
         coverage1

       ↳ config2
         agent2
          ↳ sequencer2
            driver2
            monitor2
         coverage2

       ↳ top_virtual_seq (created in run_phase)
          ↳ agent1_seq
             ↳ transaction1
          ↳ agent2_seq
             ↳ transaction2
```

给定两个接口模板，没有寄存器模型，并且每个 agent 组件均设置 `agent_has_env = yes` 时，生成代码的结构如下：

```
testbench (module)
 ↳ test harness (module)
   ↳ interface1 (interface instance)
     interface2 (interface instance)
     DUT (module instance)

 ↳ top_config (object)
   test (object)
      top_env
       ↳ config1
         env1
          ↳ agent1
             ↳ sequencer1
               driver1
               monitor1
             coverage1

       ↳ config2
         env2
          ↳ agent2
             ↳ sequencer1
               driver2
               monitor2
             coverage2

       ↳ top_virtual_seq (created in run_phase)
          ↳ env1_virtual_seq
             ↳ agent1_seq
                ↳ transaction1
          ↳ env2_virtual_seq
             ↳ agent2_seq
                ↳ transaction2
```

给定三个接口模板，其中两个具有寄存器模型且设置 `agent_has_env = yes`，另一个没有寄存器模型且设置 `agent_has_env = no` 时，生成代码的结构如下：

```
testbench (module)
 ↳ test harness (module)
   ↳ interface1 (interface instance)
     interface2 (interface instance)
     interface3 (interface instance)
     DUT (module instance)

 ↳ top_config (object)
   test (object)
      top_env
       ↳ register_model

       ↳ env_config1
         env1
          ↳ config1
            agent1
             ↳ sequencer1
               driver1
               monitor1
            coverage1
            adapter1
            predictor1

       ↳ env_config2
         env2
          ↳ config2
            agent2
             ↳ sequencer2
               driver2
               monitor2
            coverage2
            adapter2
            predictor2

       ↳ config3
         agent3
          ↳ sequencer3
            driver3
            monitor3
         coverage3

       ↳ top_virtual_seq (created in run_phase)
          ↳ env1_virtual_register_seq
            env2_virtual_register_seq
            agent3_seq
             ↳ transaction3
```

11 编译和仿真

原件页码：35-36

成功运行代码生成器后，可以按以下方式运行仿真：

Cadence：

```
cd <project>/sim
./compile_ius.sh
```

Mentor：

```
cd <project>/sim
vsim -c -do "do compile_questa.do; run -all"
```

Synopsys：

```
cd <project>/sim
./compile_vcs.sh
```

Aldec：

```
cd <project>/sim
vsimsa -do compile_riviera.do
```

代码生成器创建的环境会在每个 agent 组件上运行一个简单序列，每个 agent 组件生成一个事务。在开始试用代码生成器时，可以编辑下面的 SystemVerilog 源文件，扩展此序列，使其生成多个事务并消耗一定的仿真时间。（从长远来看，更好的解决方法是在一个新的测试中设置工厂覆盖。）

```
<project>/tb/<top>/sv/<top>_env.sv
```

`<top>_env` 启动一个顶层虚拟序列，该虚拟序列会在每个 agent 组件上启动一个序列。具体实现方式取决于该 agent 组件的模板文件中的设置。当 `agent_has env = no` 时，顶层虚拟序列直接在 agent 组件上启动序列。当 `agent_has env = yes` 时，顶层虚拟序列启动一个与 `<agent>_env` 关联的虚拟序列，再由后者在 agent 组件上启动一个非虚拟序列。

顶层虚拟序列使用文件 `<top>_seq_lib.sv` 中如下形式的代码启动子序列，每个 agent 组件对应一个子序列：

```
repeat (m_seq_count)
begin
  ...
  fork
    m_<agent1>_env_seq.start(null, this);
    m_<agent2>_env_seq.start(null, this);
    m_<agent3>_env_seq.start(null, this);
    ...
  join
end
```

因此，子序列将同步步进运行，每一轮中的所有序列都结束之后，才会启动下一轮序列。使用生成的代码运行初始仿真时，可以将 `<top>_env::run_phase` 任务中的变量 `vseq.m_seq_count` 设置为大于一的事务数，也可以覆盖运行在 agent 组件上的子序列，使其生成多个事务，或者同时采用这两种方法。（无论如何，这段代码只是用于帮助入门的示例，开发测试时应当覆盖这些序列。）

12 添加用户自定义序列

原件页码：36-37

为每个 agent 组件生成一个序列 `<agent>_default_seq`。该默认序列执行单个事务。其设计思路是，可以通过扩展 `<agent>_default_seq` 来创建自己的用户自定义序列，并在测试中添加工厂覆盖，以自己的序列替换 `<agent>_default_seq`。如果 `agent_has_env = yes`，则 `<agent>_env` 具有一个 `<agent>_env_default_seq`，由它启动 `<agent>_default_seq`。

可以在接口模板文件中指定用户自定义序列的名称和位置，以及工厂覆盖：

```
agent_seq_inc     = <sequence-include-filename>
agent_factory_set = <original-classname>  <replacement-classname>

agent_env_seq_inc = <sequence-include-filename>
agent_factory_set = <original-classname>  <replacement-classname>
```

`<sequence-include-filename>` 必须是 `<inc_path>` 目录中某个文件的名称（见上文）。`<original-classname>` 必须是要被覆盖的序列的名称。`<replacement-classname>` 必须是扩展后序列的名称。

使用 `agent_seq_inc` 时，include 文件应包含一个继承自 `<agent>_default_seq` 的类。使用 `agent_env_seq_inc` 时，include 文件应包含一个继承自 `<agent>_env_default_seq` 的类。`agent_factory_set` 生成一个工厂覆盖，以用户自定义序列替换默认序列。

13 添加用户自定义寄存器序列

原件页码：37

还可以在接口模板文件中指定用户自定义寄存器序列的名称和位置：

```
reg_seq_inc       = <register-sequence-include-filename>
agent_factory_set = <original-classname>  <replacement-classname>
```

`<register-sequence-include-filename>` 必须是 `<inc_path>` 目录中某个文件的名称（见上文）。在这种特定情况下，`<original-classname>` 必须是 `<agent>_env_default_seq`，`<replacement-classname>` 必须是扩展后序列的名称。

包含的文件应当含有一个继承自 `<agent>_env_default_seq` 的类。`agent_factory_set` 生成一个工厂覆盖，以用户自定义序列替换 `<agent>_env_default_seq`。

14 时钟和复位

原件页码：37-38

顶层测试平台 `<top>_tb` 在名为 `<top>_th` 的测试支架模块中实例化 DUT，调用 `run_test` 运行 UVM 测试，并在配置数据库中设置虚拟接口。测试支架模块实例化 DUT，默认还包含用于生成时钟和复位信号的示例代码。如果在接口模板文件中指定了 `if_clock` 和 `if_reset` 端口，则这些端口将分别连接到测试支架模块的时钟和复位信号。通常需要根据 DUT 接口的具体情况修改测试支架模块中的时钟和复位生成代码。为此，可以在通用模板文件中设置 `th_generate_clock_and_reset = no` 和 `th_inc_inside_module` 变量，并提供一个 include 文件来替换自动生成的代码。测试平台和测试支架模块生成在以下文件中：

```
<project>/tb/<top>_tb/sv/<top>_tb.sv
<project>/tb/<top>_tb/sv/<top>_th.sv
```

15 对生成的代码进行最小扩展

原件页码：38-40

为了让验证环境向 DUT 发送信号，可以提供由各个 UVM 驱动器的 `run_phase` 任务执行的代码。为避免修改生成的源代码，可以在 `<inc_path>` 目录中提供将包含到驱动器中的代码片段。必须将接口模板文件中的 `driver_inc` 设置为 include 文件的名称，并在同一个模板文件中仅设置一个 `if_clock`。`driver_inc` 依赖于 `if_clock` 设置给出的时钟名称。必须确保 include 文件的名称不与任何生成文件的名称冲突。include 文件中的代码片段必须遵循以下模式：

```
task <agent_name>_driver::do_drive();
  // This is an example. Do whatever you need to do to drive the pins of the interface.

  // The transaction is named req
  // The virtual interface variable is named vif
  // The signal names are from the .tpl file for the particular interface

  // vif.cmd <= req.cmd;
  // vif.addr <= req.addr;
  // vif.wdata <= req.data;
  // @(posedge vif.clock);
endtask
```

为了让验证环境监视 DUT 接口的引脚，可以提供各个 UVM 监视器的 `run_phase` 任务代码。为避免修改生成的源代码，可以在 `<inc_path>` 目录中提供将包含到监视器中的代码片段。必须将接口模板文件中的 `monitor_inc` 设置为 include 文件的名称。必须确保 include 文件的名称不与任何生成文件的名称冲突。include 文件中的代码片段必须遵循以下模式：

```
task <agent_name>_monitor::do_mon();
  // This is an example. Do whatever you need to do to monitor the pins of the interface.

  // This task is expected to contain an infinite loop
  // The virtual interface variable is named vif
  // The signal names are from the .tpl file for the particular interface
  // The analysis transaction is named m_trans
  // The analysis port is named analysis_port

  // forever @(negedge vif.clock)
  // begin
  //   m_trans.cmd  = vif.cmd;
  //   m_trans.addr = vif.addr;
  //   m_trans.data = vif.wdata;
  //   analysis_port.write(m_trans);
  // end
endtask
```

为了让验证环境收集覆盖率信息，可以为各个覆盖率收集器组件提供覆盖点代码，覆盖率收集器是一个 UVM 订阅器。为避免修改生成的源代码，可以在 `<inc_path>` 目录中提供覆盖组代码片段，以将其包含到覆盖率收集器中。也可以包含一个 `run_phase` 任务，例如提出 objection，直到达到某个覆盖率目标。必须将接口模板文件中的 `reg_cover_inc` 或 `agent_cover_inc` 设置为 include 文件的名称：如果 agent 组件关联了寄存器块，则使用 `reg_cover_inc`；否则使用 `agent_cover_inc`。无论哪种情况，该文件都会被包含在与该 agent 组件连接的覆盖率收集器组件的覆盖组中。必须确保 include 文件的名称不与任何生成文件的名称冲突。include 文件中的代码片段必须遵循以下模式：

```
// The covergroup must be named m_cov
// The transaction received by the subscriber is named m_item

covergroup m_cov;
  option.per_instance = 1;

  // This is an example. Provide whatever coverpoints you need.
  cp_cmd:  coverpoint m_item.cmd;
  cp_addr: coverpoint m_item.addr;
  cp_data: coverpoint m_item.data {
    bins zero = {0};
    bins one  = {1};
    bins negative = { [-128:-1] };
    bins positive = { [1:127] };
    option.at_least = 16;
  }
endgroup
```

`reg_cover_inc` 的行为已修改为与 `agent_cover_inc` 一致，也就是说，include 文件应包含一个名为 `m_cov` 的完整覆盖组。为兼容较早版本代码生成器的模板文件，`reg_cover_inc` 的 include 文件仍然可以只包含覆盖点。

16 最小 include 文件的规则

原件页码：40-41

根据所涉及的类不同，有几个需要注意的特殊之处：

- 监视器的分析端口在 `new` 中实例化，而不是在 `build_phase` 中实例化。这违反了编码指南，但好处是用户可以在 include 文件中提供自己的 `build_phase` 方法。

- 订阅器（并且只有订阅器）不提供 `new` 方法。这样，用户就可以提供自己的 `new` 方法，在需要时实例化覆盖组（这是必需的，因为 SystemVerilog 要求在 `new` 中实例化覆盖组）。

指定 `driver_inc` 时，会生成一个调用 `do_drive` 的默认 `run_phase` 方法，而必须在 include 文件中重载 `do_drive`。否则，必须在 `driver_inc_inside_class` 指定的 include 文件中定义 `run_phase`，采用类内定义或 `extern` 声明的方式（在后一种情况下，还需要指定 `driver_inc_after_class` 来提供 `extern` 方法的实现）。只指定 `_after_class` include 文件而不同时指定相应的 `_inside_class` include 文件，没有多大意义。如果同时指定 `driver_inc` 和 `driver_inc_inside_class`，则后者可用于提供由 `do_drive` 调用的方法。

指定 `monitor_inc` 时，会生成一个调用 `do_mon` 的默认 `run_phase` 方法，而必须在 include 文件中重载 `do_mon`。否则，必须在 `monitor_inc_inside_class` 指定的 include 文件中定义 `run_phase`。同样，可以同时指定 `monitor_inc` 和 `monitor_inc_inside_class`。

指定 `agent_cover_inc` 时，需要在 include 文件中提供一个覆盖组。否则，如果指定 `agent_cover_inc_inside_class`，则需要提供订阅器组件的全部内容，其中可以包含覆盖组，也可以不包含。如果两者都未指定，则会生成一个默认覆盖组，以及对该覆盖组采样的 `write` 方法。`agent_cover_inc` 和 `agent_cover_inc_inside_class` 互斥。

17 在类之前、类内部和类之后包含代码

原件页码：41

上一节描述的方法允许通过包含用户自定义代码片段来扩展生成的代码，但这些代码片段必须与自动生成的 `run_phase` 和 `write` 方法相配合。还有另一种方法，允许替换组件类的全部内容，在适用时也包括阶段方法和 `write` 方法。这种替代方法需要提供更多代码，但灵活性也大得多。

另一种方法是为每个类、接口或模块提供最多三个 include 文件（在某些情况下甚至可以更多）。例如，第一个文件可以包含在类之前，第二个包含在类内部，第三个包含在类之后。可以在这些 include 文件中放入任何语法有效的代码。所包含文件的位置由通用模板文件的 `inc_path` 设置给出。可以让生成的代码包含 `` `include `` 指令，也可以将用户自定义代码以内联方式插入生成的代码中。对于某些类，还可以在特定阶段方法的开头和末尾添加代码（例如 `agent_append_to_build_phase`）。另外还有一些设置，可以禁止在类内自动生成 `extern` 方法声明以及在类后自动生成相应的方法定义，从而允许用用户自定义代码完全替换自动生成的方法。例如：

```
driver_generate_methods_inside_class = no
driver_generate_methods_after_class  = no
```

在通用模板文件中设置以下内容，即可查看生成代码中所有 include 文件的位置：

```
comments_at_include_locations = yes
```

（这是默认设置）。驱动器类自动生成的代码随后将如下所示（为简单起见，省略条件编译保护）：

文件名 `my_driver.sv`

```
// You can insert code here by setting driver_inc_before_class in file my.tpl

class my_driver extends uvm_driver #(my_tx);

  `uvm_component_utils(my_driver)

  virtual my_if vif;

  extern function new(string name, uvm_component parent);

  // You can insert code here by setting driver_inc_inside_class in file my.tpl
```

```
endclass : my_driver


function my_driver::new(string name, uvm_component parent);
  super.new(name, parent);
endfunction : new

// You can insert code here by setting driver_inc_after_class in file my.tpl
```

18 链接

原件页码：42

Easier UVM 编码指南

Easier UVM - 深入讲解

Easier UVM 代码生成器

Easier UVM 视频教程

Easier UVM 论文和海报

可在 EDA Playground 上直接运行的 Easier UVM 示例

返回完整的 UVM 资源列表：https://www.doulos.com/knowhow/systemverilog/uvm/

附录 A 在 Linux + VCS 工作服务器上落地

本附录为实施建议，不属于原文翻译。已知条件是 Linux + VCS；具体 VCS release、UVM 版本及公司工具环境脚本尚未提供，所以下列路径和示例模板名需要替换为实际值。

A.1 先确定三种不同的“成功”

- 生成成功：Perl 生成器读取模板、include 和 DUT 输入，产生 SV 骨架与仿真脚本。此步骤本身不需要 VCS 仿真许可证。
- 骨架跑通：VCS 能编译、启动 UVM 测试并按预期结束，说明编译链和基本连接可用。
- 真实 DUT 验证成功：driver 真正驱动协议，monitor 正确还原事务，scoreboard/断言能够识别故意注入的错误。默认示例打印 transaction 不能代替这些工作。

A.2 获取和保存官方生成器

从 Doulos 官方 Easier UVM 页面进入下载入口。当前入口要求填写登记信息；应由你本人按公司流程下载，不假设存在稳定的公开 ZIP 直链。将原始发行包、版本说明及校验值保存在受版本管理的工具目录。无外网服务器可通过公司批准的渠道接收发行包及离线 Perl 依赖。

建议目录职责：tools/ 保存生成器；templates/ 保存 .tpl 和 pinlist；rtl_input/ 保存所需 RTL 与 files.f；include/ 保存用户代码；生成目录只作为可重复构建产物。首次先使用官方发行包中的 minimal 示例，目录名称以解压后实物为准。

A.3 加载公司工具环境，检查前提

先运行你们已有的 VCS 环境设置脚本或工具模块命令。不要另造许可证地址，也不要在不清楚原配置时覆盖公司环境。以下命令只用于检查：

```
command -v perl vcs vlogan gcc g++ make
perl -e 'require 5.008; print "Perl $^V\n"'
perl -MFile::Copy::Recursive -e 1
perl -MFile::stat -e 1
vcs -ID
```

Perl 要求至少 5.8.0。File::Copy::Recursive 是需要核对的非核心依赖；现代 Perl 中 File::stat 通常属于核心模块，缺失时先排查 Perl 版本和 @INC。原文虽在安装段开头列出 Time::Piece，随后又明确指出 2015-06 之后不再要求它，因此只作为旧版兼容检查。vcs -ID 不能证明仿真许可证已经成功签出，仍需实际编译和运行。

A.4 无 root 安装 Perl 模块

优先使用公司已经提供的 Perl 模块。如果已有 cpanm 且允许联网，可安装到自己的用户目录：

```
cpanm --local-lib="$HOME/perl5" local::lib File::Copy::Recursive
eval "$(perl -I"$HOME/perl5/lib/perl5" -Mlocal::lib="$HOME/perl5")"
perl -MFile::Copy::Recursive -MFile::stat -e 'print "Perl modules OK\n"'
```

如果没有 cpanm 或服务器不能联网，请让 IT 提供本地 Perl 库或离线模块包，并按公司约定配置 PERL5LIB。不要把原文的 root 安装命令直接当成普通工作账号的操作前提。

A.5 先运行官方最小示例

将 minimal 示例复制到自己的可写工作目录。在模板所在目录运行生成器；下面的 /path/to/... 是占位路径，clkndata.tpl 是文档入门示例名，要按实际包中的文件替换：

```
perl /path/to/easier_uvm_gen.pl -h
perl /path/to/easier_uvm_gen.pl -m common.tpl clkndata.tpl
```

-t 可省略。common.tpl 默认名为 common.tpl；prefix 默认 top；project 默认 generated_tb。project 必须是简单目录名，而不是带斜杠的路径。先 cd 到合适工作目录，再生成。初次调通不要用 -c 自动跳过所有警告；先逐条弄清警告原因。

每次重新生成都会更新生成目录。长期修改要保存在模板、include 文件或可重复的后处理脚本中，避免仅手改生成结果。可以使用 Git 保存输入及发行包版本。

A.6 检查并运行生成的 VCS 脚本

```
cd generated_tb/sim
head -n 1 compile_vcs.sh
grep -nE 'vcs|vlogan|uvm|UVM|ntb_opts|simv|files' compile_vcs.sh
./compile_vcs.sh
```

原文的 VCS 入口是 <project>/sim/compile_vcs.sh。先查看脚本使用哪种 shell、如何选择 UVM、是否已同时编译并启动 simv，再运行。旧发行包可能依赖 csh/tcsh；按脚本 shebang 和公司平台处理，不要无条件用 bash 硬跑。2016 年脚本的工具开关不能假定与所有当前 VCS release 一致。

UVM 只选择一套：优先采用公司已经验证的 VCS/UVM 组合。使用 VCS 内建 UVM 时，根据本地 vcs -help 和公司 flow 选择对应选项；常见示意为 -ntb_opts uvm-1.2，但实际可用版本要核对。如果使用外部 UVM，则按公司 flow 编译它。不要同时选择内建 UVM 并再手动编译另一份 uvm_pkg.sv。

A.7 再替换为真实 DUT

- common.tpl：填写真实 DUT 模块名 dut_top、相对当前生成工作目录的 dut_source_path、inc_path、dut_pfile，以及 prefix/project 等设置。
- RTL/filelist：准备完整依赖与编译顺序。生成器会复制 dut_source_path 下的文件；其中 files.f 只列 DUT 及所需选项，不要再列一份 UVM 基类库。嵌套 include 或外部 package 的路径需单独核对。
- 每类接口一份 .tpl：定义 agent_name、trans_item、trans_var、if_port、时钟/复位标识以及 agent 模式。pinlist 将接口变量映射到真实 DUT 端口。
- 用户代码：实现 driver 的 do_drive 或 run_phase、monitor 的 do_mon 或 run_phase，以及 sequence。默认 driver 打印收到的事务不意味着已经驱动 DUT。
- 检查与覆盖：接入记分板/参考模型、断言、覆盖率和异常场景。Syosil scoreboard 是可选额外依赖，最小骨架 smoke 不必引入它。

先调通一个接口和一个最小测试，再增加多 agent、多实例、寄存器层和复杂参考模型。你之前生成的 DV_SVA_ASSERT 库可以通过 interface/assertion module 或 bind 方式加入，不会由 Easier UVM 自动理解并生成协议断言。

A.8 调通的验收清单

- 生成阶段：没有未处理的模板警告；生成目录结构和 DUT 实例端口映射与预期相符。
- 编译阶段：无 package 重复定义、无缺失 include/模块/类，UVM 版本和工具链一致，许可证可正常签出。
- 基础运行：选中的 UVM test 确实启动；run_phase、sequence、driver/monitor 的基本连接可观察；测试能按预期结束。
- 真实协议：波形上时钟/复位正确、接口确实活动；monitor 还原的事务与驱动一致；检查器对正确场景通过、对故意违规失败；覆盖点确实被触发。

A.9 接入真实工程还需准备的信息

VCS 的具体版本及公司 UVM 选型；DUT 顶层模块和端口声明；时钟/复位极性与时序；需要几个接口/agent；事务字段与握手协议；RTL files.f；可用的参考模型或期望结果。缺少这些输入时可以生成学习骨架，但不能替你完成真实协议驱动和功能检查。

本译本没有连接或修改你的工作服务器，没有下载执行生成器，也没有运行 VCS。以上步骤和原文提供的脚本入口应在你们的实际工具环境中进行 smoke 验证。

实施参考来源

Doulos 官方下载入口：https://www.doulos.com/knowhow/systemverilog/uvm/easier-uvm/easier-uvm-coding-guidelines/download-the-easier-uvm-coding-guidelines-and-code-generator/

Doulos 入门教程：https://www.doulos.com/knowhow/systemverilog/uvm/easier-uvm/easier-uvm-code-generator/easier-uvm-code-generator-tutorial-1/

用户目录 Perl 模块安装参考：https://metacpan.org/pod/App::cpanminus 与 https://metacpan.org/pod/local::lib。VCS 具体版本开关以公司安装版本的帮助和支持文档为准。